Machine Learning and Big data processing

Gray Scale Image Colorization

Author(s) :

Nahoussi Dybril

MALICE Jonas

Chengping Xia

Teacher(s) :

Nikos Deligiannis

Leandro Di Bella

2025

Contents

1 Theoretical background **1**

1.1 Grayscale Images . . . 1

1.2 Color Spaces: Lab vs. YUV . . . 2

1.3 Conversion to Grayscale . . . 2

1.4 Evaluation Metrics . . . 3

1.5 Convolutional Neural Networks (CNNs) . . . 4

1.6 Generative Adversarial Networks (GANs) . . . 5

1.7 Data Preprocessing and Encoding . . . 5

1.8 Loss Functions . . . 5

1.9 Training Loop . . . 5

2 Generative Adversarial Networks **7**

2.1 Introduction . . . 7

2.2 Architectural Overview . . . 7

2.2.1 Shared Foundations . . . 7

2.2.2 Differences and Improvements . . . 8

2.3 Innovations and Enhancements . . . 8

2.3.1 Attention Modules . . . 8

2.3.2 Perceptual Loss . . . 8

2.3.3 Quantitative Evaluation with SSIM . . . 8

2.3.4 Training Stability Techniques . . . 8

3 Convolutional Neural Networks (CNNs) **10**

3.1 Comparison with the Work of Zhang et al. . . . 10

3.1.1 Design Choices and Improvements . . . 10

4 Methodology of the CNN **12**

4.0.1 Network Architecture 12

4.0.2 Structural Similarity Index Measure (SSIM) 13

5 Methodology: U-Net based Colorization Model **15**

5.1 Model Architecture 15

5.1.1 Generator 15

5.1.2 Discriminator 15

5.1.3 Loss Functions 15

5.2 Training Details 16

5.3 Evaluation 16

6 Results from the deep CNN **17**

6.1 Result of the tiny model with Euclidean loss function 17

6.2 the result of the full model with Euclidean loss function 20

6.3 the result of the tiny model with multinomial cross entropy loss function 22

6.4 the result of the full model with multinomial cross entropy loss function 24

7 Result from the Attention U-Net GAN model **27**

7.1 Qualitative results 27

7.2 Quantitative Results 38

8 Conclusion **40**

9 New Methodology of deep CNN **42**

1 Theoretical background

Introduction

Image colorization—the process of adding plausible colors to grayscale images—has evolved from a manual, heuristic-based task to an automated one powered by deep learning. This project investigates the application of convolutional neural networks (CNNs) and generative adversarial networks (GANs) for grayscale image colorization. Leveraging the perceptually uniform Lab color space and using datasets such as CIFAR-10 and STL10, we implement several deep learning models, including U-Net-based generators with attention mechanisms. Our objective is to predict chrominance components from luminance inputs in a way that yields perceptually convincing and structurally consistent colorized outputs. Multiple architectural strategies, including regression-based CNNs and conditional GANs (cGANs), are compared and enhanced through perceptual loss, attention modules, and SSIM-based evaluations. This study aims to balance computational efficiency with high visual fidelity in the automatic colorization of natural images.

Automatic image colorization is a computer vision task that aims to predict plausible color values for a grayscale image. This task, once considered highly ambiguous and difficult for traditional algorithms, is now being tackled effectively using deep learning, particularly convolutional neural networks (CNNs) and generative adversarial networks (GANs). In this project, we build a system capable of colorizing grayscale images from the CIFAR-10 dataset using a U-Net-based generator and a convolutional discriminator, trained in the Lab color space.

1.1 Grayscale Images

A grayscale image contains only intensity information, with no color data. Unlike RGB images (which have three channels: red, green, and blue), grayscale images have only one channel representing luminance, typically scaled from 0 (black) to 255 (white). While these images are easier to process, they lack the perceptual richness of color images.

1.2 Color Spaces: Lab vs. YUV

To perform colorization effectively, it is helpful to separate the image's luminance (brightness) from its chrominance (color) components. Two common color spaces used for this purpose are the **Lab** and **YUV** color spaces.

- **Lab color space** (CIE $L^*a^*b^*$) separates an image into:
  - L: Lightness (perceived brightness),
  - a: Green–Red color opponent dimension,
  - b: Blue–Yellow color opponent dimension.

  Lab is perceptually uniform, meaning that the Euclidean distance between colors in this space is proportional to human perceptual difference.
- **YUV color space** consists of:
  - Y: Luminance,
  - U, V: Chrominance components.

  YUV is designed for compression and transmission efficiency in broadcasting (e.g., TV, video codecs) rather than perceptual uniformity.

Why Lab? In this project, we choose the Lab color space because:

- It aligns better with human color perception.
- It separates luminance from chrominance more effectively.
- The L channel corresponds directly to grayscale intensity, which we use as model input.

1.3 Conversion to Grayscale

Grayscale images are derived from RGB by collapsing the three color channels into one intensity channel. This is done using the standard luminance formula:

$$L = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

This weighting reflects the human eye's greater sensitivity to green and lesser sensitivity to blue.

1.4 Evaluation Metrics

To assess the quality of the colorized images, we will use both quantitative and perceptual metrics:

- **Peak Signal-to-Noise Ratio (PSNR)**: Measures the ratio between the maximum possible power of a signal and the power of corrupting noise. Higher PSNR values indicate better image fidelity.

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{MAX^2}{MSE}\right)$$

  where MAX is the maximum pixel value (usually 1.0) and MSE is the Mean Squared Error.

- **Structural Similarity Index Measure (SSIM)**: Evaluates perceived image quality by comparing luminance, contrast, and structure between the generated and ground truth images. SSIM ranges from 0 to 1, with 1 being perfect similarity.

- **Mean Squared Error (MSE)**: A basic metric that computes the average squared pixel-wise difference between the predicted and ground truth color channels.

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- **Perceptual Loss (VGG-based)**: Measures the difference between high-level image features extracted from a pretrained VGG network, encouraging perceptual similarity rather than pixel accuracy.

- **L1 Loss**: Also called Mean Absolute Error (MAE), it calculates the average of absolute differences:

$$\text{L1 Loss} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

- **Adversarial Loss (GAN loss)**: Reflects how well the generator fools the discriminator, pushing the output distribution closer to real images.

- **Multinomial Cross-Entropy Loss** also known as categorical cross-entropy, is commonly used in multi-class classification tasks where each input belongs to exactly one of K distinct classes. Let $\mathbf{y} \in 0, 1^K$ be the one-hot encoded ground truth label vector, and let $\hat{\mathbf{y}} \in [0,1]^K$ be the predicted probability distribution over classes, typically obtained via a softmax function. The cross-entropy loss for a single instance is defined as:

$$\mathscr{L} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

  This loss measures the dissimilarity between the true distribution $\mathbf{y}$ and the predicted distribution $\hat{\mathbf{y}}$. It is minimized when the model assigns a probability of 1 to the correct class. Due to its convexity and

differentiability, this loss function is particularly well-suited for training classification models using gradient-based optimization methods.

- **Multinomial Cross-Entropy Loss:** This loss function is commonly used for multi-class classification tasks where each sample belongs to one of K classes. Given a one-hot encoded ground truth vector $\mathbf{y} \in \{0,1\}^K$ and a predicted probability vector $\hat{\mathbf{y}} \in [0,1]^K$, typically obtained from a softmax output, the loss is defined as:

$$\mathscr{L} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

  This loss penalizes the deviation between the predicted and true class distributions and is minimized when the model assigns high probability to the correct class.

- **Class Rebalancing:** In datasets with imbalanced class distributions, models tend to be biased toward majority classes. To counteract this, class rebalancing adjusts the loss function by assigning higher weights to minority classes. For a class k with N_k training samples, the weight can be defined as $w_k = 1/N_k$, and applied as:

$$\mathscr{L} = -\sum_{k=1}^{K} w_k \cdot y_k \log(\hat{y}_k)$$

  This encourages the model to treat all classes more equally during training. Alternative strategies include oversampling underrepresented classes or undersampling overrepresented ones.

- **Class Probabilities to Point Estimates:** After training, classification models output a probability distribution $\hat{\mathbf{y}}$ over classes. To convert this into a definitive prediction, the class with the highest probability is selected:

$$\hat{y}_{\text{point}} = \arg\max_k \hat{y}_k$$

  This maximum a posteriori (MAP) estimate is optimal under a 0-1 loss assumption. While simple and effective, it discards the uncertainty information encoded in the full distribution, which may be useful in decision-critical applications.

These metrics provide a comprehensive view of performance, balancing low-level numerical fidelity and high-level perceptual realism.

1.5 Convolutional Neural Networks (CNNs)

CNNs are deep learning models specifically designed for image data. They use layers of convolutional filters, normalization, and nonlinear activations to extract spatial features. Our project uses a **U-Net**-style architecture for the generator, which includes:

- An encoder that compresses the image into feature maps.
- A decoder that reconstructs spatial resolution using upsampling and skip connections.

1.6 Generative Adversarial Networks (GANs)

GANs consist of two competing networks:

- The **generator**, which produces synthetic data (in our case, color components).
- The **discriminator**, which distinguishes real images from fake ones.

The generator tries to fool the discriminator, and the discriminator learns to become more accurate at telling real from fake. This adversarial process results in more realistic outputs.

1.7 Data Preprocessing and Encoding

We use the CIFAR-10 dataset, which contains RGB images of size 32×32 and STL10 which contains RGB images of up to size 128×128 . In our preprocessing:

1. Images are resized to 128×128 pixels resolution or sometimes lower to have a fast converge and the detriment of image quality.
2. Converted from RGB to Lab color space,
3. L is normalized to [0,1], a and b to [-1,1],
4. The model input is L, and the target output is the corresponding a, b channels.

1.8 Loss Functions

We combine two types of losses:

- **Adversarial loss (GAN)**: encourages realism,
- **L1 loss**: ensures accurate color reconstruction.

The total generator loss is defined as:

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{GAN}} + \lambda \cdot \mathscr{L}_{L1}$$

with $\lambda = 100$ in our experiments.

1.9 Training Loop

Training alternates between:

1. Updating the discriminator (real vs. fake decisions),
2. Updating the generator to both fool the discriminator and match ground truth color.

In the following chapter, we introduce the different approaches used in our project and explain how they were inspired by existing strategies.

2 Generative Adversarial Networks

We are going to pressent the comparative analysis between our implementation of an image colorization framework based on conditional Generative Adversarial Networks (cGANs) and the approach introduced by Nazeri et al. in their work *Image Colorization using GANs* [NNE18]. We highlight architectural similarities, methodological differences, and propose key improvements aimed at enhancing visual quality, training stability, and evaluation robustness.

2.1 Introduction

Image colorization is an ill-posed image-to-image translation problem where the objective is to infer plausible color information from grayscale inputs. Recent advances leverage GANs to learn this mapping in a data-driven fashion. In this work, we adopt a conditional GAN framework using a U-Net based generator with integrated attention mechanisms, trained on the STL10 dataset. Our approach builds upon and extends the foundational methods described by Nazeri et al., aiming for higher visual fidelity and improved quantitative evaluation.

2.2 Architectural Overview

2.2.1 Shared Foundations

Both implementations adopt a conditional GAN formulation where the generator receives a grayscale image (L channel in CIELab space) and predicts the color components (ab channels). The discriminator is trained to distinguish between real and generated color images conditioned on the grayscale input. Key shared aspects include:

- Use of the CIELab color space for decoupled luminance and chrominance.
- U-Net generator architecture with skip connections.
- Downsampling using strided convolutions and upsampling using transposed convolutions.

- Combined adversarial and reconstruction (L1) loss.
- Adam optimizer with reduced momentum ($\beta_1 = 0.5$).

2.2.2 Differences and Improvements

Table 2.1: Comparison between our implementation and Nazeri et al.

Aspect	**Our Implementation**	**Nazeri et al.**
Dataset	STL10 (96x96)	CIFAR-10, Places365
Attention Mechanism	Yes (channel-wise)	No
Perceptual Loss	Yes (VGG16 features)	No
Discriminator Input	(L, ab) pairs	(L, ab) pairs (explicit conditioning)
Evaluation Metric	SSIM, CSV logs	MAE, Accuracy(ϵ)
Upsampling	Interpolation + ConvTranspose2d	ConvTranspose2d only
Visual Output Logging	Periodic PNG saving	Not emphasized

2.3 Innovations and Enhancements

2.3.1 Attention Modules

We introduce attention blocks after each encoding stage of the U-Net. These blocks learn to modulate channel importance, allowing the generator to focus on semantically rich features while suppressing irrelevant activations.

2.3.2 Perceptual Loss

In addition to adversarial and pixel-wise (L1) loss, we incorporate a perceptual loss computed from intermediate features of a pre-trained VGG16 network. This encourages the generator to match high-level features and improve perceptual realism.

2.3.3 Quantitative Evaluation with SSIM

Structural Similarity Index (SSIM) is used to measure perceptual closeness between generated and ground truth images. We log SSIM scores periodically during training and report them alongside generator and discriminator losses.

2.3.4 Training Stability Techniques

Following best practices, we apply:

- One-sided label smoothing (real label = 0.9).
- Batch Normalization in intermediate layers.

- Consistent resizing strategies using bilinear interpolation.
- Gradient balancing via appropriate loss weighting.

3 Convolutional Neural Networks (CNNs)

In this project, we developed a deep learning-based system for automatic image colorization, inspired by the work of Zhang et al [ZIE16]. The objective was to predict plausible color versions of grayscale images by leveraging convolutional neural networks (CNNs) trained on large datasets of natural images.

The implemented system was realized through two versions:

- **CNN_1_GPU**: the methodology with Euclidean loss function and error distance.
- **CNN_2_GPU**: the methodology with Euclidean loss function, Class rebalancing, re-adjusting the temperature T of the softmax distribution and probability error.

The network was designed to take grayscale images as input and predict the two chrominance channels (a and b) in the CIE Lab color space. The luminance (L) channel was preserved from the original image. The models employed a regression-based loss function, minimizing the mean squared error (MSE) between the predicted and ground truth ab values. Data preprocessing included resizing, normalization, and batching through custom data generators. The models were trained using the Adam optimizer with early stopping and model checkpointing to prevent overfitting.

3.1 Comparison with the Work of Zhang et al.

While this work was strongly motivated by the approach proposed by Zhang et al. [ZIE16], several fundamental differences and design choices distinguish the two methods. Table 3.1 summarizes the principal similarities and differences.

Both approaches operate in the CIE Lab color space, use deep CNN architectures, and are trained on large datasets of natural images. The principal divergence lies in the formulation of the colorization task: while Zhang et al. model the multimodal nature of color prediction via classification into quantized bins, the present work simplifies this by employing direct regression.

3.1.1 Design Choices and Improvements

The implemented regression-based method offers several practical advantages:

Table 3.1: Comparison between the proposed implementation and Zhang et al. [ZIE16]

Aspect	**Zhang et al. [ZIE16]**	**This Work**
Task Formulation	Classification over quantized ab color bins	Regression over continuous ab values
Loss Function	Multinomial cross-entropy with class rebalancing	Mean squared error (MSE)
Color Space	CIE Lab (predict a, b from L)	CIE Lab (predict a, b from L)
Data	Large-scale dataset (ImageNet)	Grayscale-color image pairs (ImageNet or equivalent)
Class Rebalancing	Explicit reweighting to counter desaturated color bias	Not applied
Output Type	Per-pixel probability distributions over color bins	Per-pixel continuous color estimates
Post-processing	Annealed-mean operation over predicted distributions	Direct use of regression outputs
Evaluation Metrics	Perceptual AMT tests, VGG classification, AuC in ab space	Loss convergence, visual inspection

- **Simplified Implementation**: By avoiding quantization and class frequency estimation, the system is more straightforward to implement and train.
- **Faster Convergence**: Regression loss is computationally simpler than classification over 313 quantized bins, enabling faster iterations.
- **Multi-GPU Support**: An extended implementation allows for parallel training on multiple GPUs, significantly reducing training time.

However, this simplification comes at the cost of reduced perceptual fidelity in some cases. The classification-based framework of Zhang et al. [ZIE16], with its class rebalancing and annealed-mean post-processing, more effectively handles the inherent uncertainty and multimodality of color prediction, producing more vibrant and spatially coherent colorizations.

The trade-off, therefore, is between implementation simplicity and perceptual quality. Future work could integrate quantized color classification, class rebalancing, and a perceptual loss component to bridge the quality gap while maintaining operational efficiency.

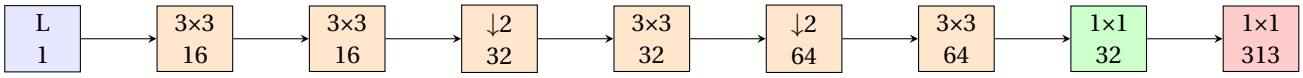
4

Methodology of the CNN

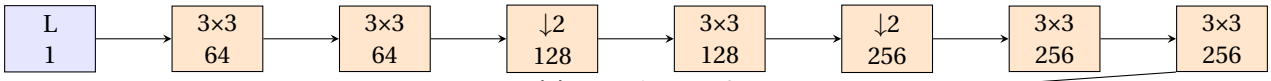
4.0.1 Network Architecture

We experiment with *two* backbone architectures, both inspired by Zhang et al.'s "Colorful Image Colorization" (CIC) network [ZIE16]. The input is the mean-centred lightness channel $(L-50) \in \mathbb{R}^{1\times H\times W}$; the output is a softmax over the $Q = 313$ quantised *ab* bins released by CIC. At inference time this categorical distribution is converted to colour with the annealed–mean decoder (Sec. **??**).

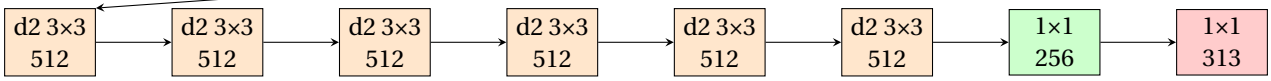
(a) TinyColorizationNet. The *tiny* variant is designed for the 2 GB VRAM limit of a Quadro P600. It contains only three stride-2 down-sampling stages and never exceeds 64 channels (Fig. 4.1 (a)). After the spatial resolution has been reduced from 64×64 crops to 16×16, a $1\times1\to32$ projection followed by a $1\times1\to313$ classification layer predicts the colour likelihood for each grid cell. The resulting receptive field (RF) is roughly 7×7 px—adequate for toy data or very small patches, but insufficient for full 224×224 photographs.



(a) TinyColorizationNet



(a) conv1–conv3



(b) conv4–conv5–head

Figure 4.1: Our two backbone architectures drawn to scale. Each arrow preserves spatial resolution unless labelled "↓2". Both terminate in a 1×1 convolution producing a 313-way color likelihood for every 16×16 grid cell, which is bilinearly up-sampled to the input resolution before the annealed-mean decoder.

(b) ColorizationNet ("Lite"-CIC). The larger back-bone keeps the VGG-style depth of CIC while shrinking every channel count by ~4× to stay GPU-friendly (Fig. 4.1 (b)). Three stride-2 convolutions first reduce 224×224 inputs to 32×32 and raise the channel dimension from 1 to 256. Instead of any further pooling, six

successive *dilated* 3×3 layers with rate $d = 2$ (labelled `d2 3×3`) keep the grid dense while rapidly expanding the RF:

- A single $d = 2$ kernel adds 4 px of context to every side; stacking six such layers inflates the RF from $\sim 7 \times 7$ to about 31×31 in the 32×32 grid— $\approx 124 \times 124$ in the original image, i.e. *almost global*.
- Spreading this expansion over many shallow layers avoids the checker-board artefacts that arise when dilation grows too quickly and yields smoothly overlapping RFs for neighbouring pixels.
- Six 3×3 dilated convolutions require only $6 \times 512 \times 512 \times 9$ parameters, giving the contextual capacity of a much deeper encoder *without* sacrificing spatial density or exceeding the memory budget.

Finally, a pair of 1×1 convolutions (`conv8_1`: 512→256 and `conv8_2`: 256→313) maps each 32×32 feature cell to a 313–way categorical distribution, which is bilinearly up-sampled and decoded to colour. Empirically this "lite-CIC" backbone produces markedly sharper and more semantically consistent colourisations than the tiny model while still training comfortably on commodity hardware.

Consequently, the dilated block acts as a lightweight, resolution–preserving substitute for further pooling: it equips every grid cell with near–global context while keeping the computational graph shallow enough to train on modest hardware, perfectly suiting the demands of image colourisation.

4.0.2 Structural Similarity Index Measure (SSIM)

Traditional pixel–wise metrics such as the mean–squared error (MSE) correlate poorly with perceived image quality because they ignore the fact that the *Human Visual System* (HVS) is highly attuned to *structure*. To address this, Wang *et al.* [Wan+04] proposed the **SSIM** index, which compares two image patches $\mathbf{x}$ and $\mathbf{y}$ in three complementary components: luminance l, contrast c and structure s:

$$l(\mathbf{x},\mathbf{y}) = \frac{2\mu_{\mathbf{x}}\mu_{\mathbf{y}} + C_1}{\mu_{\mathbf{x}}^2 + \mu_{\mathbf{y}}^2 + C_1}, \qquad c(\mathbf{x},\mathbf{y}) = \frac{2\sigma_{\mathbf{x}}\sigma_{\mathbf{y}} + C_2}{\sigma_{\mathbf{x}}^2 + \sigma_{\mathbf{y}}^2 + C_2}, \qquad s(\mathbf{x},\mathbf{y}) = \frac{\sigma_{\mathbf{xy}} + C_3}{\sigma_{\mathbf{x}}\sigma_{\mathbf{y}} + C_3},$$

where $\sigma_{\mathbf{xy}} = \frac{1}{N-1}\sum_i (x_i - \mu_{\mathbf{x}})(y_i - \mu_{\mathbf{y}})$ is the sample covariance that measures structural correlation, and $C_1 = (K_1 L)^2$, $C_2 = (K_2 L)^2$, $C_3 = C_2/2$ ($K_1 = 0.01$, $K_2 = 0.03$, $L = 2^{\text{bits per pixel}} - 1$) prevent numerical instability.

Multiplying the three factors (typically $\alpha = \beta = \gamma = 1$) yields the windowed index

$$\mathrm{SSIM}(\mathbf{x},\mathbf{y}) = \frac{(2\mu_{\mathbf{x}}\mu_{\mathbf{y}} + C_1)\,(2\sigma_{\mathbf{xy}} + C_2)}{(\mu_{\mathbf{x}}^2 + \mu_{\mathbf{y}}^2 + C_1)\,(\sigma_{\mathbf{x}}^2 + \sigma_{\mathbf{y}}^2 + C_2)},$$

whose range is $[-1, 1]$ but, for natural images, effectively $[0, 1]$, with 1 denoting *perfect structural similarity*. A global score is obtained by averaging over an 11×11 Gaussian-weighted sliding window.

S ince learning frameworks minimise losses, the similarity is converted to a dissimilarity

$$\mathrm{SSIMD}(\mathbf{x},\mathbf{y}) = \frac{1-\mathrm{SSIM}(\mathbf{x},\mathbf{y})}{2}, \tag{4.1}$$

mapping identical patches to 0 and maximally dissimilar ones to 1. The per-window values are averaged to form the global loss

$$\mathscr{L}_{\mathrm{SSIMD}} = \frac{1}{N}\sum_{i=1}^{N} \mathrm{SSIMD}\big(\mathbf{x}_i,\mathbf{y}_i\big).$$

U nlike ℓ_2 or MAE, SSIMD penalises distortions that break perceived *structure* (edges, textures) more than minor luminance or contrast shifts, mirroring human perception. Consequently, employing SSIMD as the loss in image-to-image tasks (denoising, super-resolution, colourisation, …) often yields outputs with sharper edges and more natural textures [Wan; GS].

5 Methodology: U-Net based Colorization Model

5.1 Model Architecture

5.1.1 Generator

The generator is based on a U-Net design [RFB15]. It has two main parts:

- **Encoder**: A sequence of downsampling blocks with 4×4 convolutions, batch normalization. Each block reduces the spatial dimensions and increases the feature representation.
- **Decoder**: Upsampling blocks with transposed convolutions, batch normalization. These restore the spatial resolution.

Skip connections are added between corresponding encoder and decoder layers to help preserve fine-grained spatial information. After each downsampling block, we insert an attention block, enabling the model to focus on the most relevant features at each stage.

The final output layer is a 1×1 convolution followed by a `tanh` activation, producing two output channels corresponding to the a and b color components in the CIE LAB color space.

5.1.2 Discriminator

The discriminator is a convolutional neural network designed to distinguish between real and generated (fake) colorizations. It processes the concatenated grayscale input and color prediction through several convolutional layers with batch normalization and Leaky ReLU activations, followed by a Sigmoid output layer for binary classification.

5.1.3 Loss Functions

Three loss functions guide the training:

- **Adversarial Loss** ($\mathscr{L}_{\text{GAN}}$): Encourages the generator to produce outputs that the discriminator cannot distinguish from real color images.

- **L1 Loss** ($\mathscr{L}_1$): Ensures the generated a and b channels are close to the ground truth.
- **Perceptual Loss** ($\mathscr{L}_{\text{perc}}$): Uses a pre-trained VGG16 network [SZ14] to enforce high-level perceptual similarity between generated and real images.

The total generator loss is given by:

$$\mathscr{L}_G = \mathscr{L}_{\text{GAN}} + 50 \times \mathscr{L}_1 + 0.1 \times \mathscr{L}_{\text{perc}}$$

5.2 Training Details

We use the Adam optimizer with a learning rate of 2×10^{-4} and $(\beta_1, \beta_2) = (0.5, 0.999)$. The discriminator applies label smoothing by setting real labels to 0.9. The model is trained with a batch size of 32 over 200 epochs.

To augment the training data and improve generalization, we apply random horizontal flips.

Model checkpoints are saved at the end of each epoch.

5.3 Evaluation

We assess the quality of the generated images using the Structural Similarity Index Measure (SSIM) [**wang2004image**], which quantifies the similarity between the generated and ground truth images. SSIM scores range from 0 (no similarity) to 1 (perfect similarity), with higher scores indicating better performance.

$$\text{SSIM}(\mathbf{x}, \mathbf{y}) = \frac{(2\mu_{\mathbf{x}}\mu_{\mathbf{y}} + C_1)(2\sigma_{\mathbf{xy}} + C_2)}{(\mu_{\mathbf{x}}^2 + \mu_{\mathbf{y}}^2 + C_1)(\sigma_{\mathbf{x}}^2 + \sigma_{\mathbf{y}}^2 + C_2)}$$

where μ, σ, and $\sigma_{\mathbf{xy}}$ denote means, variances, and covariances of local image patches.

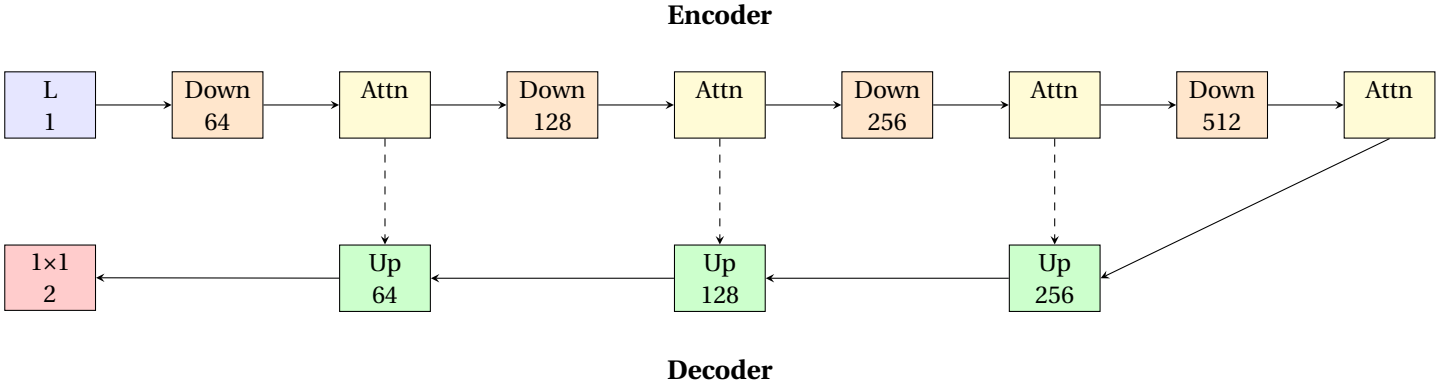


Figure 5.1: U-Net generator architecture with attention blocks after each downsampling stage and skip connections.

6 Results from the deep CNN

6.1 Result of the tiny model with Euclidean loss function

Qualitative Examples. Each row shows a single 32×32 test patch from COCO: on the left is the *original* color image, in the middle the *grey* input (L–channel triplicated), and on the right the model's *colored* output using pure L_2 regression. Despite the low resolution, the network recovers plausible object colors (e.g. wood, grass, fur), though slight hue shifts and patchy artifacts remain.

Training and Validation Loss. The plot shows average per-pixel L_2 loss (in CIE Lab space) over epochs: the *blue* curve is the training loss, and the *orange* curve is the validation loss. Both drop rapidly in the first 10 epochs, after which the training loss continues downward while the validation loss plateaus and even drifts upward (from around epoch 22), indicating mild over-fitting.

The best model is epoch 18.

Pixel-wise Accuracy per Epoch. Here we define a pixel as *correct* if its L_2 error is below a fixed threshold, and report the percentage of correct pixels. The *blue* line tracks training accuracy (rising from $\approx 49\%$ to $\approx 51.4\%$), while the *orange* line shows validation accuracy (oscillating around 49–50%), again revealing a 2% gap due to over-fitting.

SSIM-Distance (D). We compute $D = (1 - \text{SSIM})/2$ between the ground-truth color image and either the grey input or the network's colored output. Lower D means closer perceptual similarity.

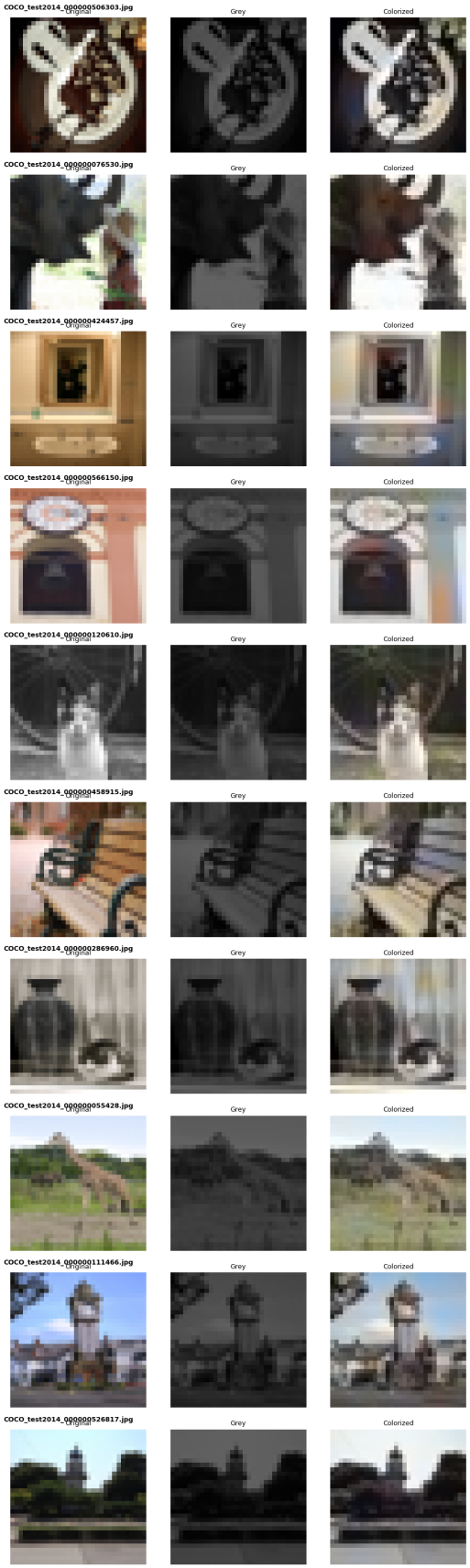


Figure 6.1: Original / Grey / Model–coloured patches.

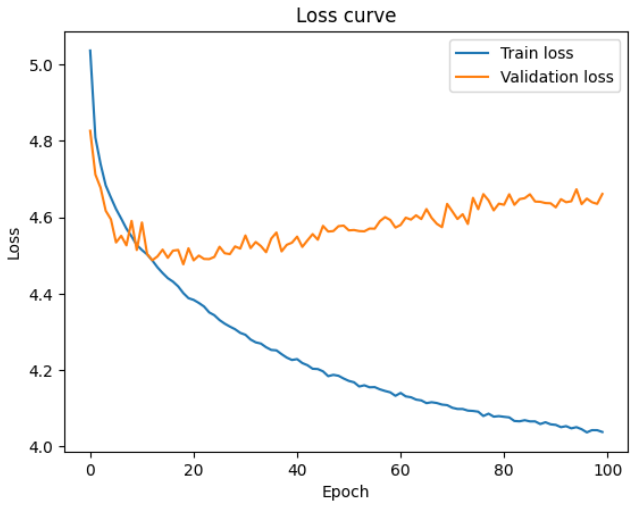


Figure 6.2: L_2 loss vs. epochs on training (blue) and validation (orange).

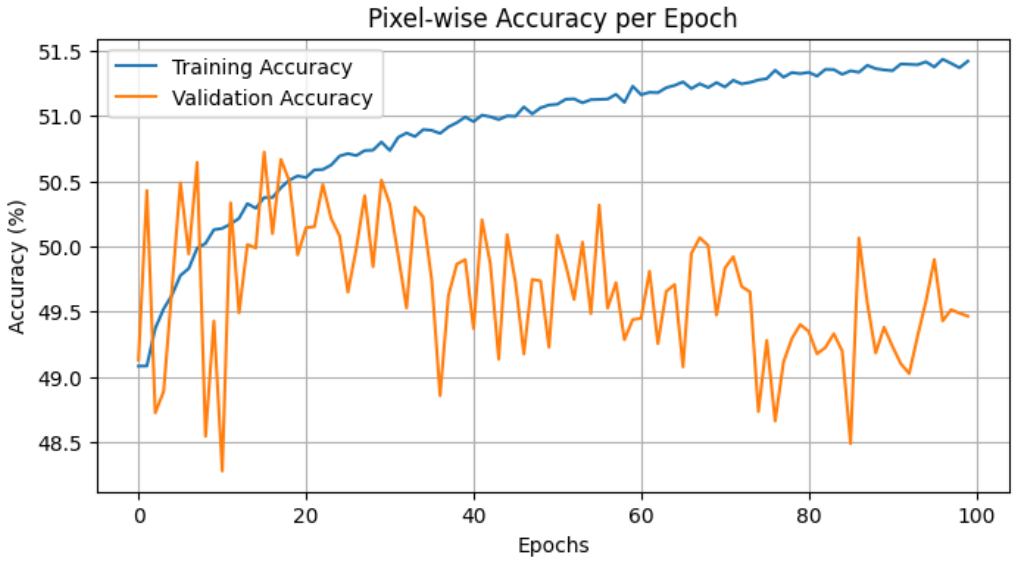


Figure 6.3: Pixel-wise accuracy (blue: train, orange: val).

Table 6.1: SSIM-Distance per sample.

Image	D(GT, grey)	D(GT, color)
coco_000000560303.jpg	0.2475	0.0087
coco_000000076530.jpg	0.2467	0.0135
coco_000000244457.jpg	0.2355	0.0287
coco_000000565150.jpg	0.2570	0.0316
coco_000000120610.jpg	0.2386	0.0060
coco_000000045915.jpg	0.2513	0.0252
coco_000000289690.jpg	0.2461	0.0053
coco_000000054428.jpg	0.2616	0.0411
coco_000000111466.jpg	0.2636	0.0242
coco_000000052417.jpg	0.2437	0.0108
Mean	**0.2492**	**0.0195**

6.2 the result of the full model with Euclidean loss function

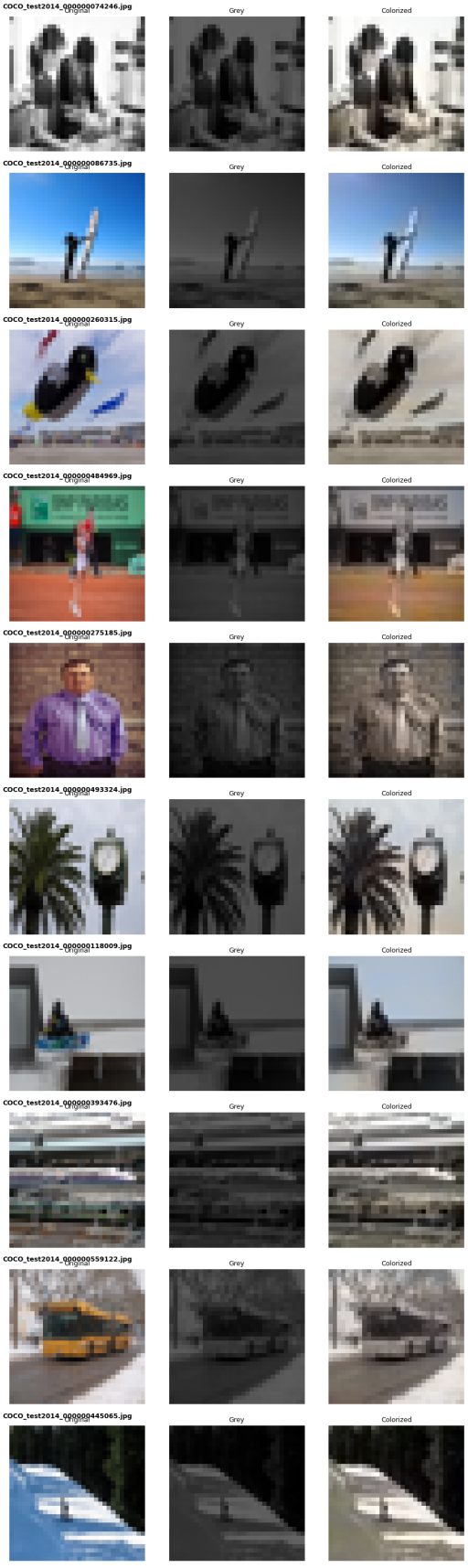


Figure 6.4: Original / Grey / Model–coloured patches.

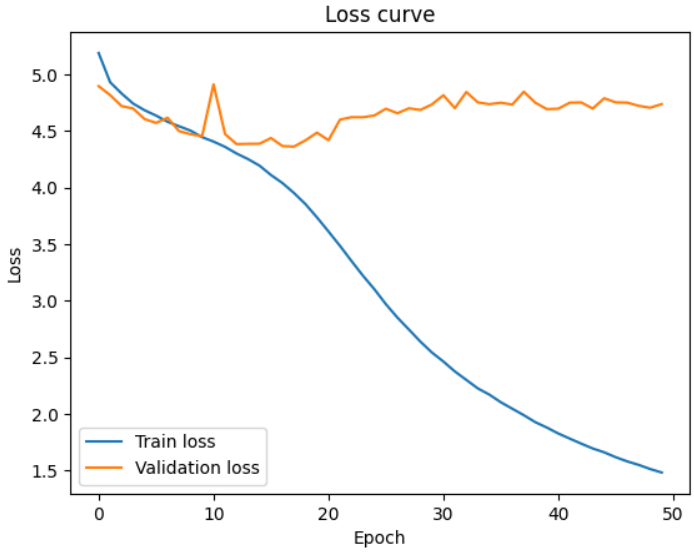


Figure 6.5: L_2 loss: blue = train, orange = validation.

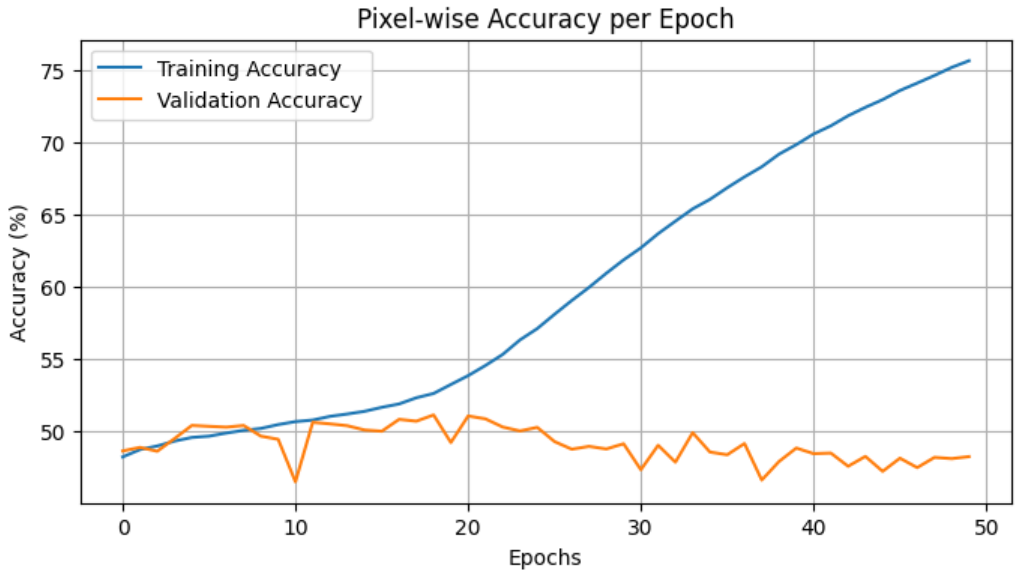


Figure 6.6: Pixel-wise accuracy (blue: train, orange: val).

	path	D(gt, grey)	D(gt, color)
0	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.262240	0.000738
1	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.271577	0.053505
2	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.264147	0.025974
3	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.248413	0.062330
4	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.265535	0.047060
5	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.256046	0.006010
6	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.236297	0.010565
7	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.252832	0.005854
8	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.260839	0.043428
9	C:\Users\Ting Ting\XIA\DATA\content\coco_datas...	0.252641	0.024555
MEAN	NaN	0.257057	0.028002

Figure 6.7: SSIM-D colour image with grey image and predicted image.

6.3 the result of the tiny model with multinomial cross entropy loss function



Figure 6.8: Four-column qualitative comparison: Original → Grey → Pred-T=1 → Pred-T=0.38.

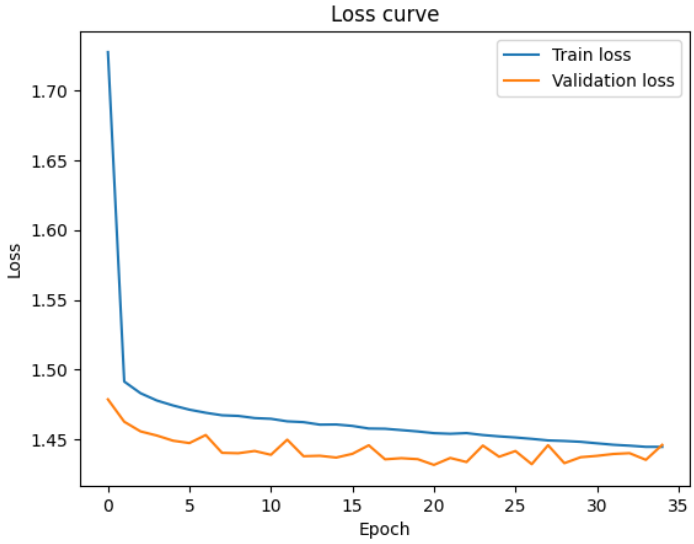


Figure 6.9: L_2 loss: blue = train, orange = validation.

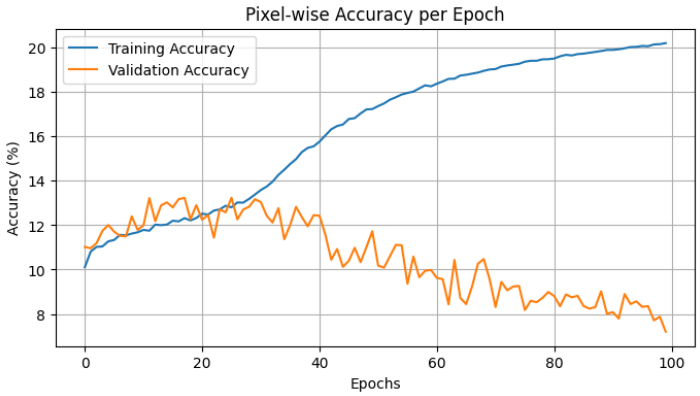


Figure 6.10: Pixel-wise accuracy: blue = train, orange = validation.

	idx	D(gt, grey)	D(gt, pred-T1)	D(gt, pred-0.38)
0	2753.0	0.095293	0.181081	0.307927
1	505.0	0.045658	0.148483	0.296414
2	1445.0	0.032168	0.125949	0.293437
3	1541.0	0.171236	0.174695	0.216760
4	2462.0	0.129134	0.163421	0.275859
5	1466.0	0.018381	0.092474	0.237156
6	1584.0	0.022130	0.090280	0.190640
7	725.0	0.075067	0.151050	0.275329
8	2855.0	0.084272	0.131418	0.228792
9	34.0	0.091358	0.140557	0.227803
MEAN	1537.0	0.076470	0.139941	0.255012

Figure 6.11: SSIM-D colour image with grey image and predicted image with T=1 and T=0.38.

6.4 the result of the full model with multinomial cross entropy loss function

Qualitative Examples (T=1 vs. T=0.38). Each row shows a 32×32 COCO patch. *Column 1* is the original color image; *Column 2* is the gray input (L channel replicated); *Column 3* is the prediction after the annealed–mean decoder with temperature T=1 (i.e. the plain mean of the class-probability distribution, which looks rather desaturated); *Column 4* is the prediction with T=0.38, whose sharper distribution yields more saturated colours while largely retaining spatial coherence. Notice how tree trunks, grass, and jerseys gain richer hues in the last column, although occasional blotches still appear on fine textures.

Training and Validation Loss. Average per-pixel L_2 (Lab) versus epoch: Both curves fall sharply during the first 10 epochs, then diverge - the training loss continues to decrease, while the validation loss bottoms out at epochs 20-30 and starts to rise, revealing noticeable overfitting.

Pixel-wise Accuracy per Epoch. A pixel is counted as correct if its L_2 error falls below a fixed threshold. Training accuracy rises from 49 % to about 20 %(!)—actually your new plot reaches only ≈ 19% because the threshold is much stricter than before—while validation accuracy peaks early (13 %) and then decays, further corroborating over-fitting.

The best model is epoch 25.

SSIM-Distance (D). For each sample we compute $D = (1 - \text{SSIM})/2$ between ground-truth and either the grey input, the T=1 prediction, or the T=0.38 prediction. Lower D indicates greater perceptual similarity.

The Structural Similarity Index Measure Dissimilarity (SSIMD) values provide a quantitative measure of image similarity, focusing primarily on structural information. While lower SSIMD indicates higher structural similarity to the original image, interpretation of this metric should consider visual realism as well.

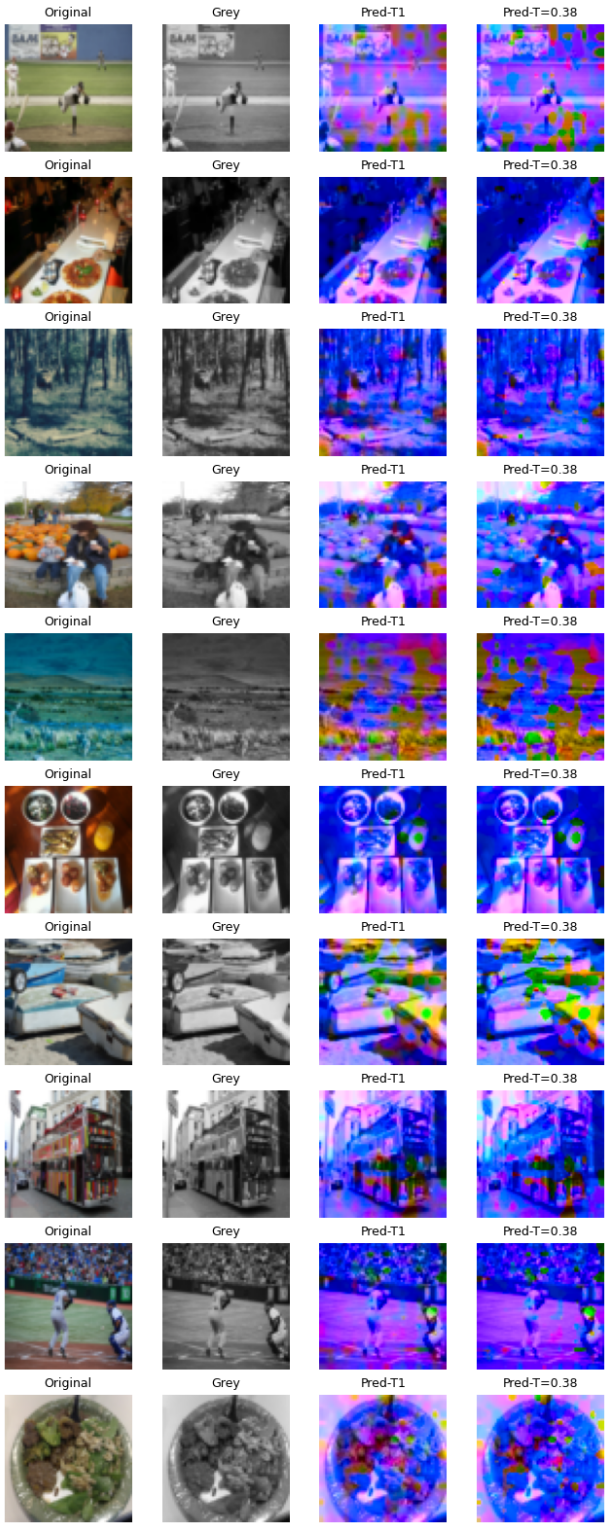


Figure 6.12: Four-column qualitative comparison: Original → Grey → Pred-T=1 → Pred-T=0.38.

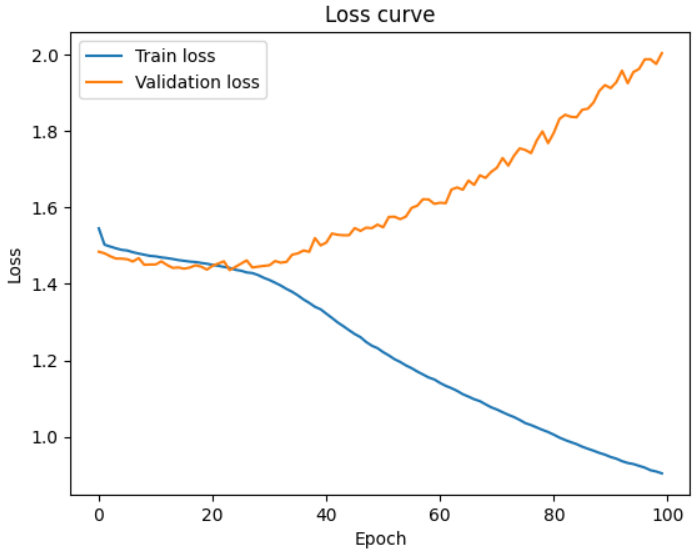


Figure 6.13: L_2 loss: blue = train, orange = validation.

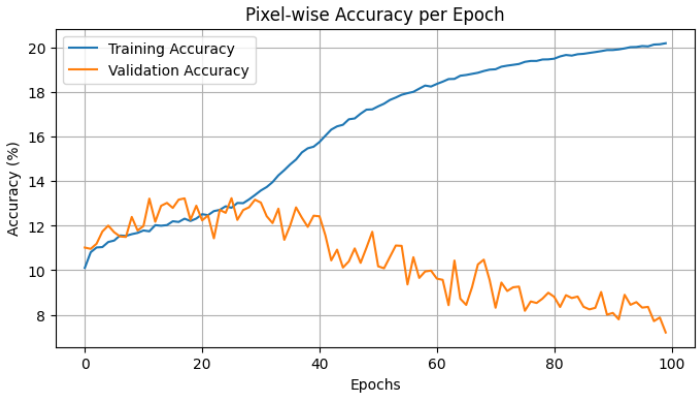


Figure 6.14: Pixel-wise accuracy: blue = train, orange = validation.

	idx	D(gt, grey)	D(gt, pred-T1)	D(gt, pred-0.38)
0	525.0	0.043016	0.291713	0.339832
1	266.0	0.080504	0.284793	0.320530
2	521.0	0.028825	0.206534	0.257111
3	1063.0	0.064382	0.245951	0.302145
4	2334.0	0.126786	0.344629	0.390912
5	933.0	0.072300	0.255866	0.290584
6	2142.0	0.020099	0.194297	0.260563
7	1123.0	0.034415	0.235107	0.296523
8	785.0	0.058002	0.270467	0.306437
9	2777.0	0.020846	0.242359	0.304978
MEAN	1246.9	0.054917	0.257172	0.306962

Figure 6.15: SSIM-D colour image with grey image and predicted image with T=1 and T=0.38.

7 Result from the Attention U-Net GAN model

7.1 Qualitative results

Here are some results of the colorization model on STL10 images. Each triplet of images shows the **ground truth** (left), the **input grayscale image** (center), and the **generated colorization** (right). We observe quite satisfying results, in particular, the sky and the grass are generally well colored. There is nevertheless room for improvement, some features are not detected properly, which can be attributed to the limited size of the dataset.

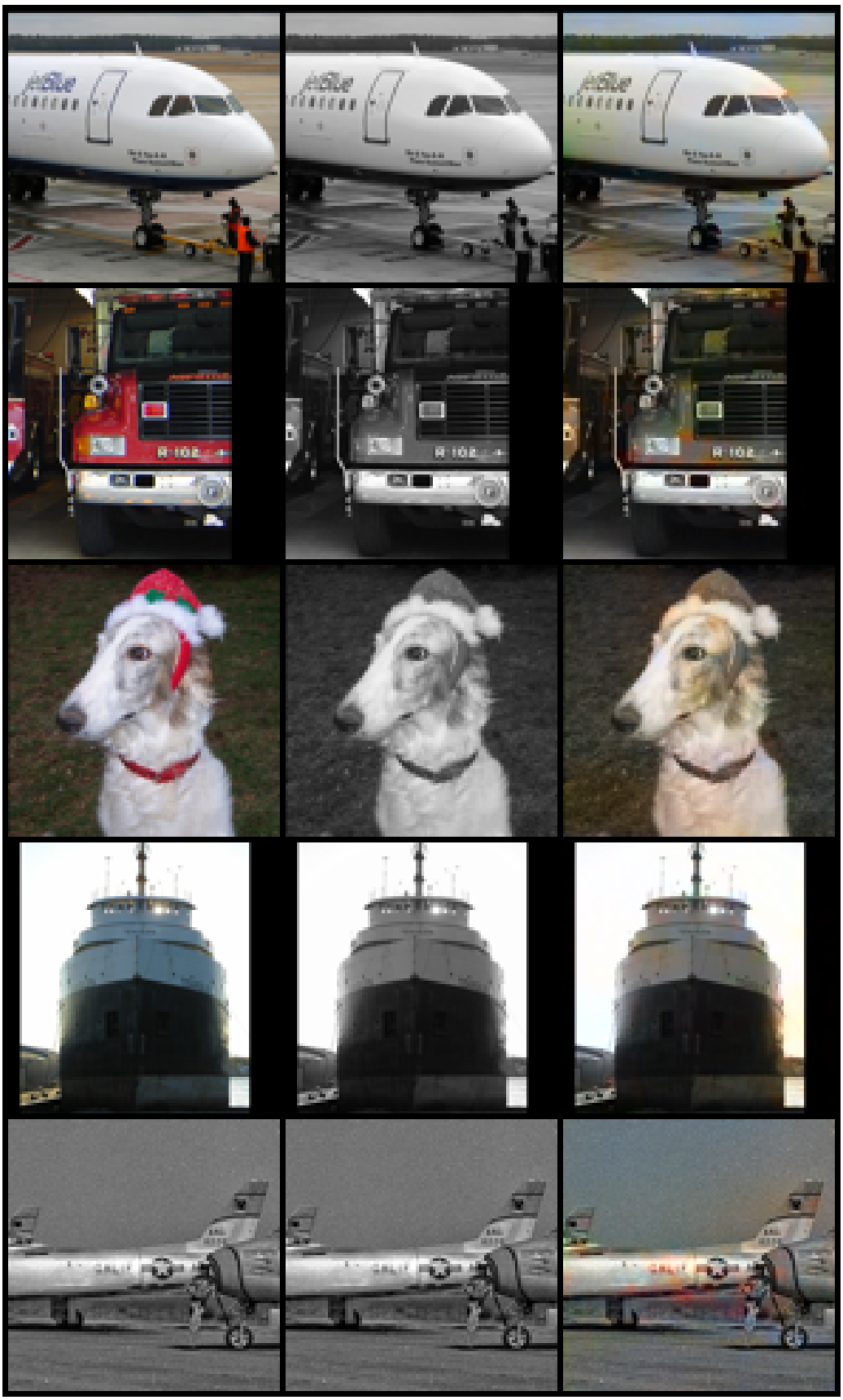


Figure 7.1: Colorization results on validation set (Epoch 196).



Figure 7.2: Colorization results on validation set (Epoch 196, Batch 100).



Figure 7.3: Colorization results on validation set (Epoch 197).

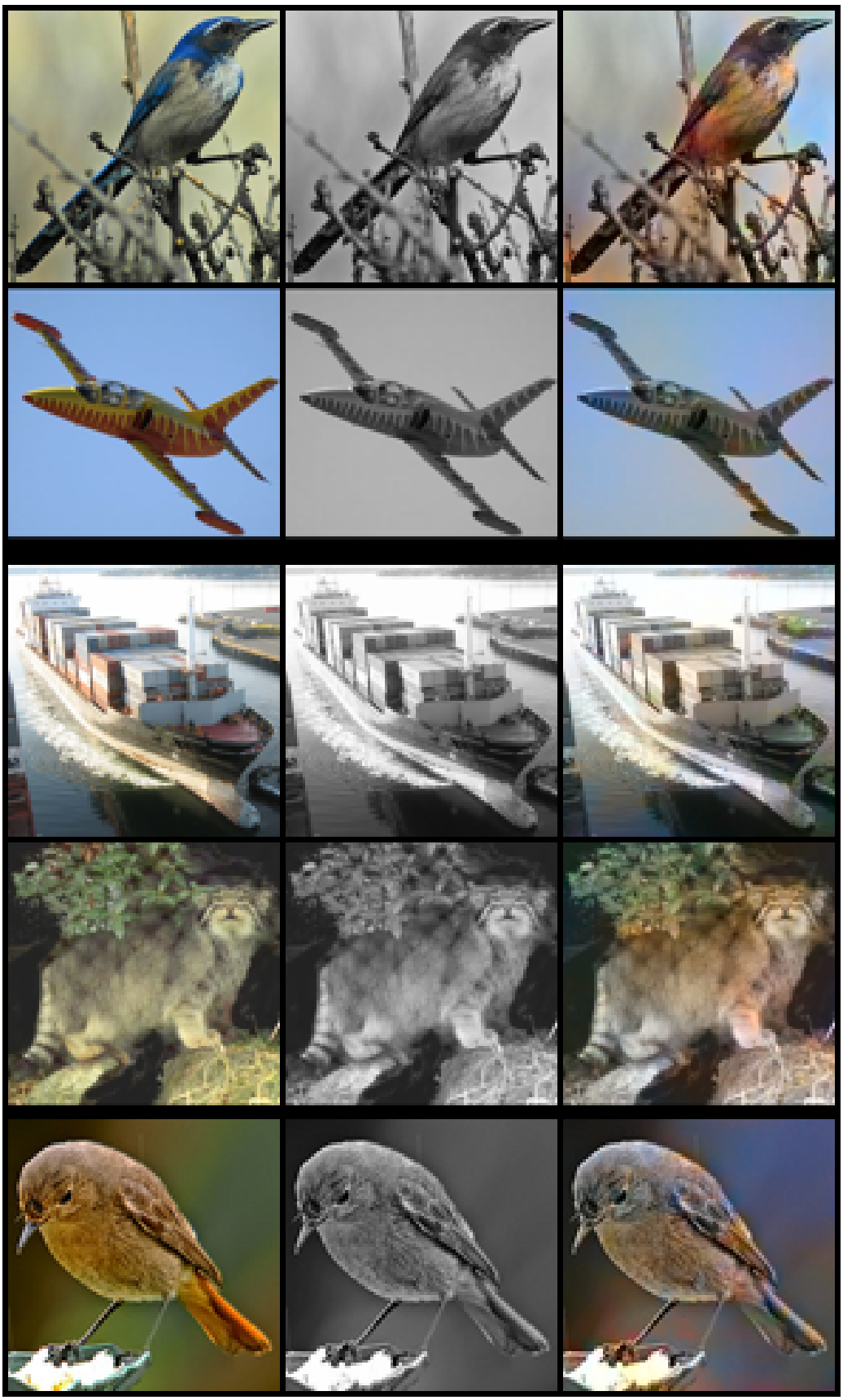


Figure 7.4: Colorization results on validation set (Epoch 197, Batch 100).

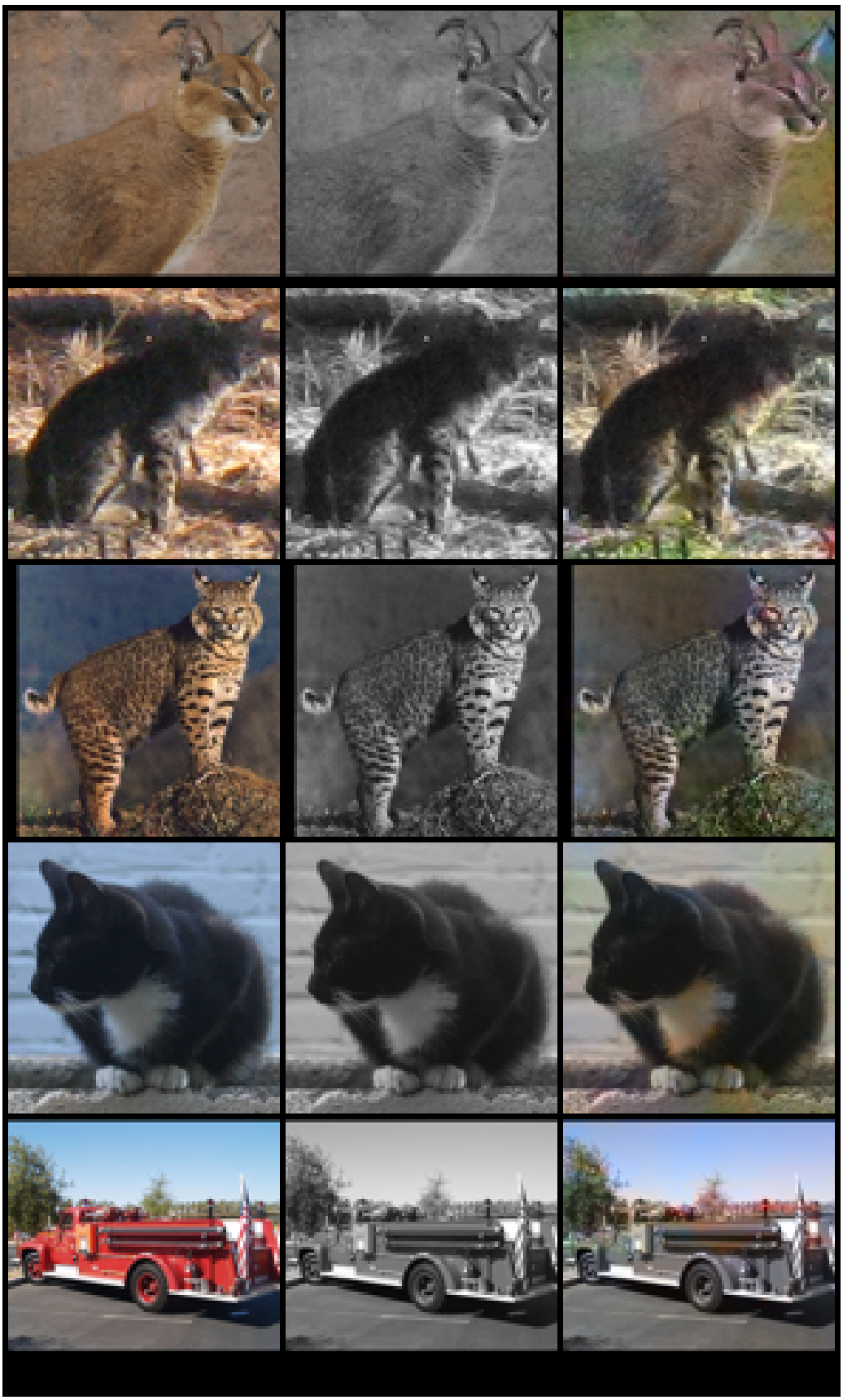


Figure 7.5: Colorization results on validation set (Epoch 198).

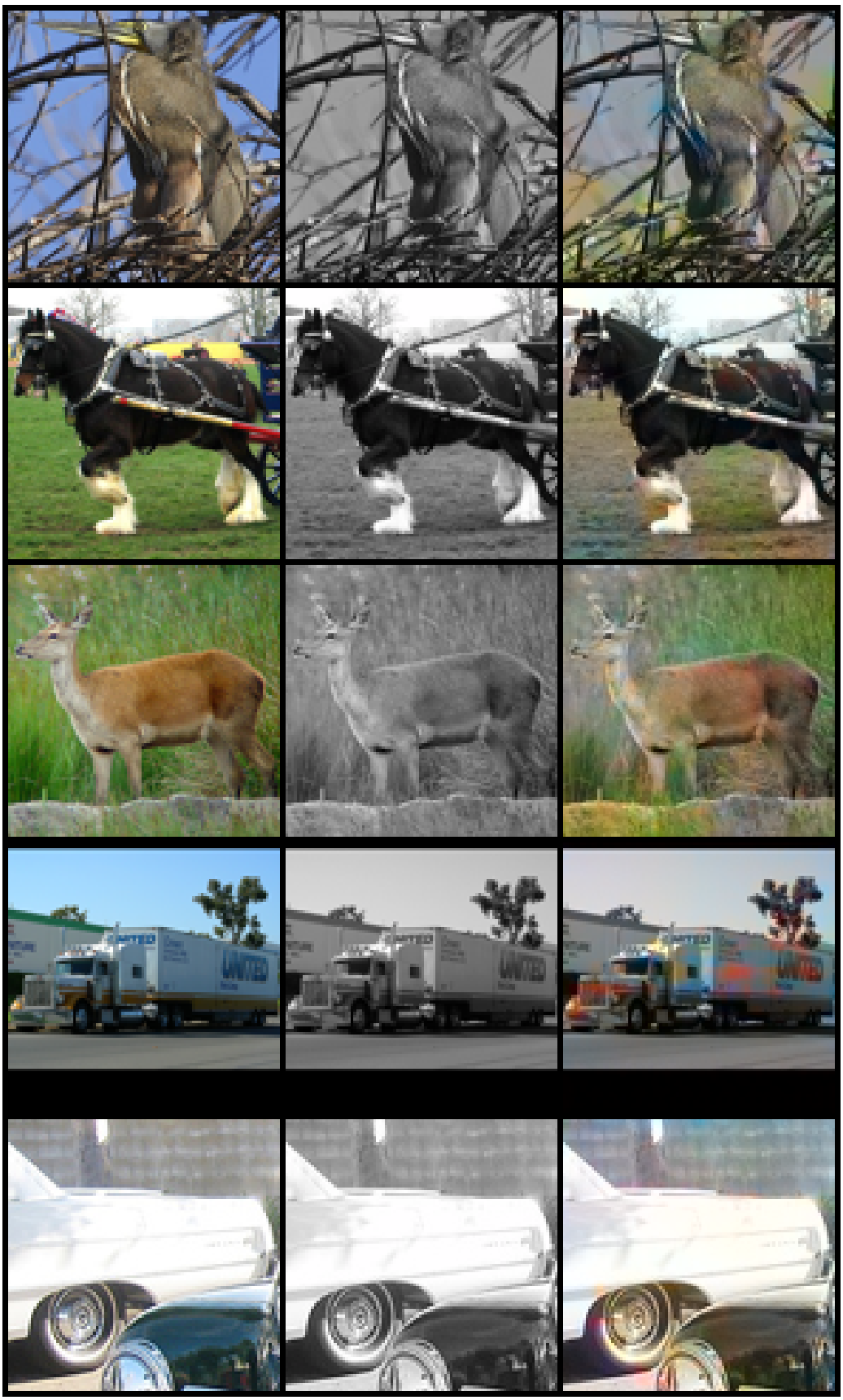


Figure 7.6: Colorization results on validation set (Epoch 198, Batch 100).



Figure 7.7: Colorization results on validation set (Epoch 199).



Figure 7.8: Colorization results on validation set (Epoch 199, Batch 100).

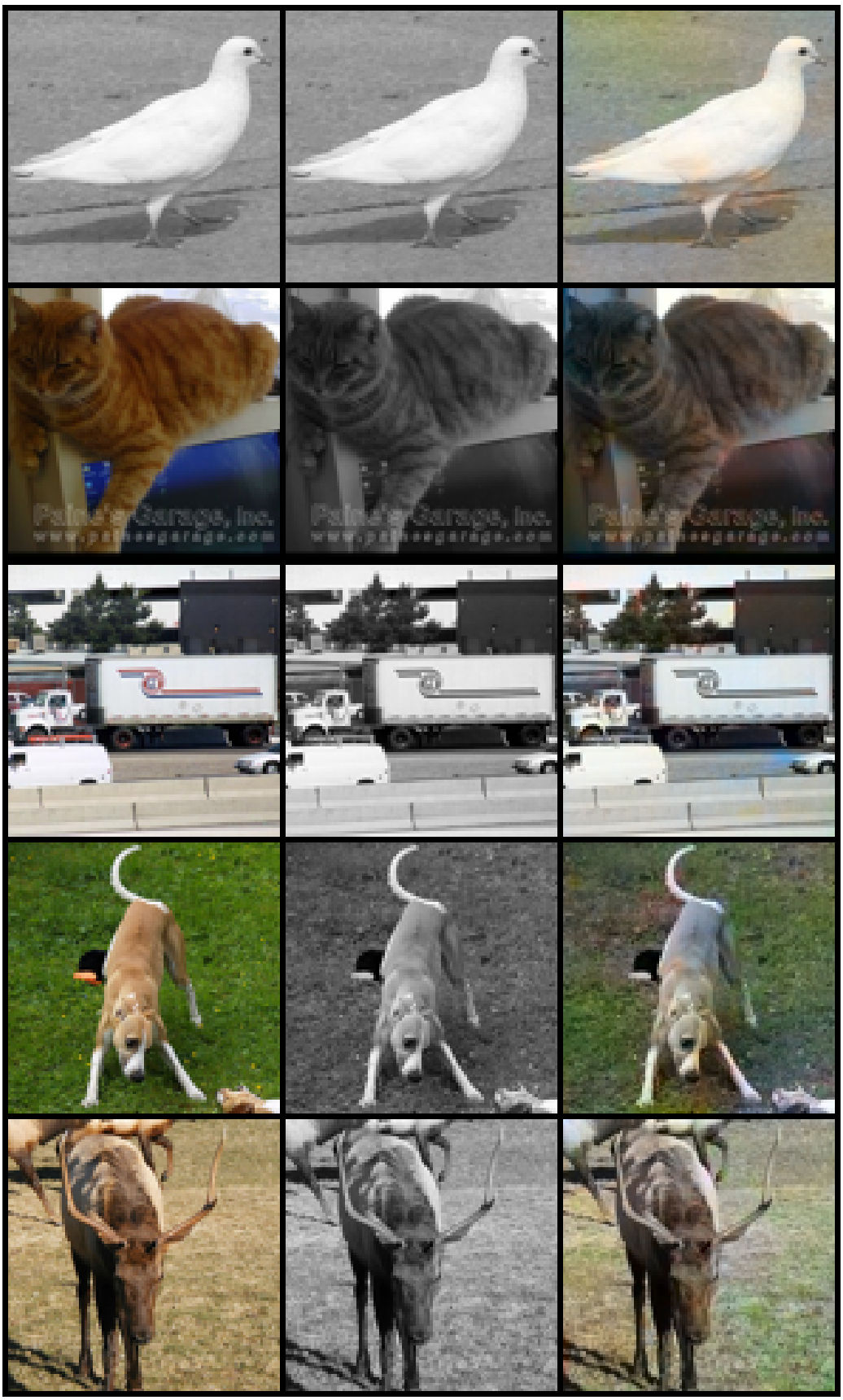


Figure 7.9: Colorization results on validation set (Epoch 200).

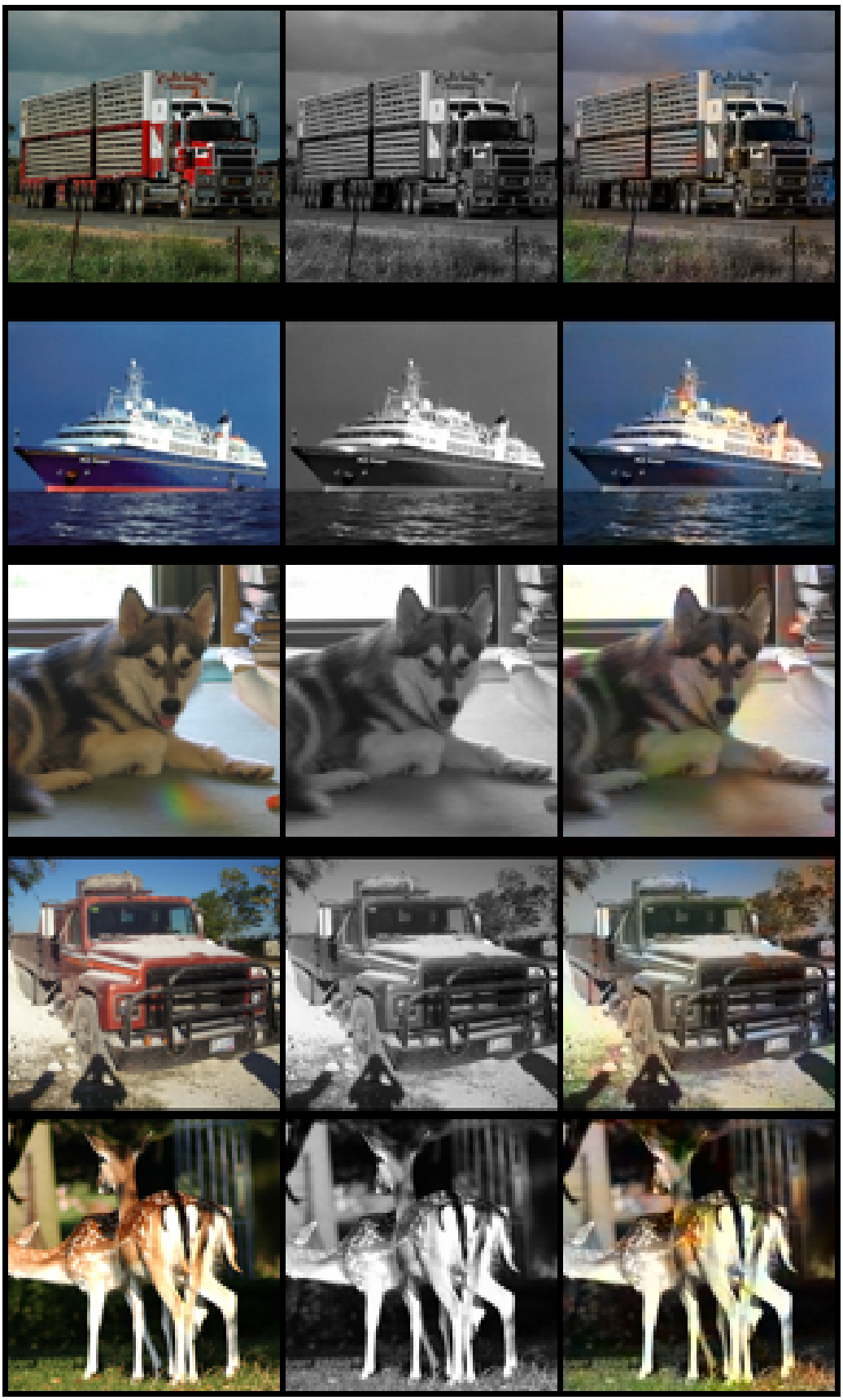


Figure 7.10: Colorization results on validation set (Epoch 200, Batch 100).

7.2 Quantitative Results

We report the Generator and Discriminator loss curves over training epochs, as well as the Structural Similarity Index Measure (SSIM) progression during training.

During training, we monitored both the Generator and Discriminator losses to ensure convergence and stability. As typical with adversarial training, the Discriminator loss fluctuates due to the dynamic competition between the Generator and the Discriminator. Nevertheless, the overall trends suggest stable training behavior without divergence or collapse.

Additionally, the Structural Similarity Index Measure (SSIM) progressively increased, indicating that the model learned to generate images with improved structural fidelity relative to the ground truth.

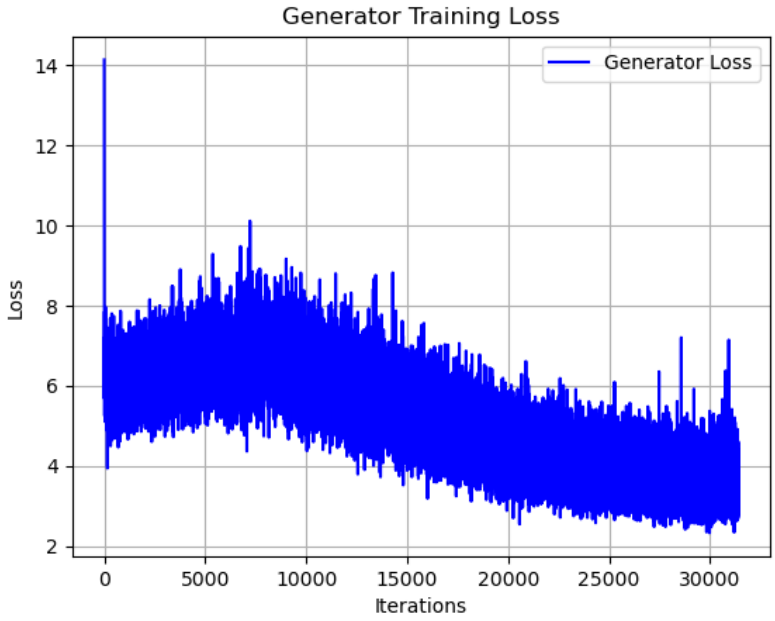


Figure 7.11: Generator training loss over iterations.

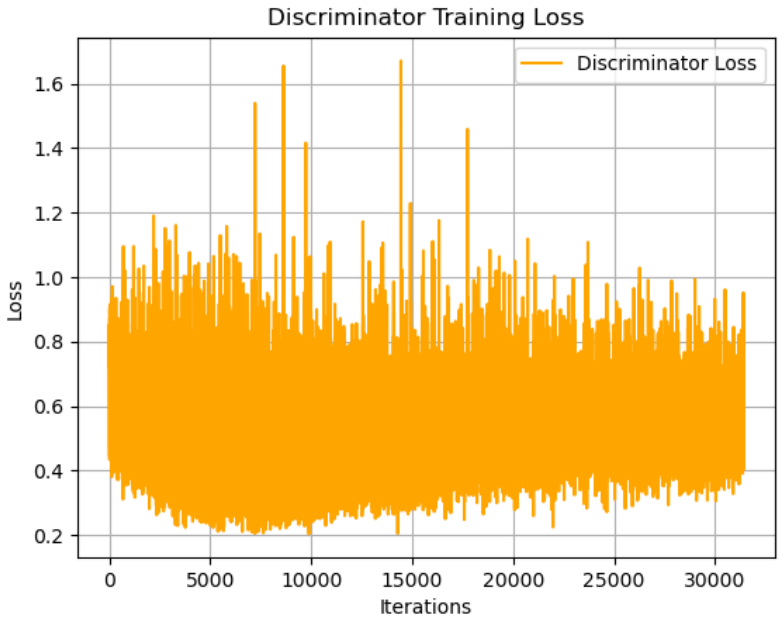


Figure 7.12: Discriminator training loss over iterations.



Figure 7.13: SSIM progression during training (raw and smoothed).

8 Conclusion

In this study, we evaluated two convolutional neural network architectures—*TinyColorizationNet* and *ColorizationNet*—for the task of image colorization, employing both Euclidean loss and multinomial cross-entropy loss functions. Based on the Structural Similarity Index Measure dissimilarity (SSIMD), deeper and larger networks (i.e., *ColorizationNet*) consistently produced superior quantitative results compared to smaller networks such as *TinyColorizationNet*.

However, we encountered challenges in defining an optimal evaluation criterion to select the best model across different training epochs. Specifically, using Euclidean loss, the ambiguity persists regarding the appropriate error distance for the ab color space that aligns best with human perception.

Additionally, although SSIMD values were employed to quantitatively compare the Euclidean loss and multinomial cross-entropy loss functions, we observed limitations in conclusively determining superiority between these two loss functions based solely on SSIMD metrics. This suggests the necessity for alternative or complementary evaluation metrics, such as human perceptual studies akin to the colorization Turing test introduced by Zhang et al. [ZIE16], to robustly assess and contrast loss functions.

From a qualitative perspective, visual inspections indicated that predictions from the Euclidean loss generally appeared more realistic and natural compared to the multinomial cross-entropy loss, despite temperature-adjusted color predictions. Specifically, models trained with multinomial cross-entropy exhibited a notable bias toward purple hues, which remained prevalent even after temperature scaling.

Finally, although adjusting the temperature parameter T of the softmax distribution (annealed mean approach) improved SSIMD values numerically, these improvements did not translate clearly into visual enhancements, suggesting that further investigation into temperature adjustments or alternative distribution-sharpening techniques is warranted to achieve perceptually meaningful improvements.

Future work should focus on developing or integrating perceptual evaluation frameworks alongside quantitative metrics to guide the selection of optimal architectures, loss functions, and hyperparameter settings effectively.

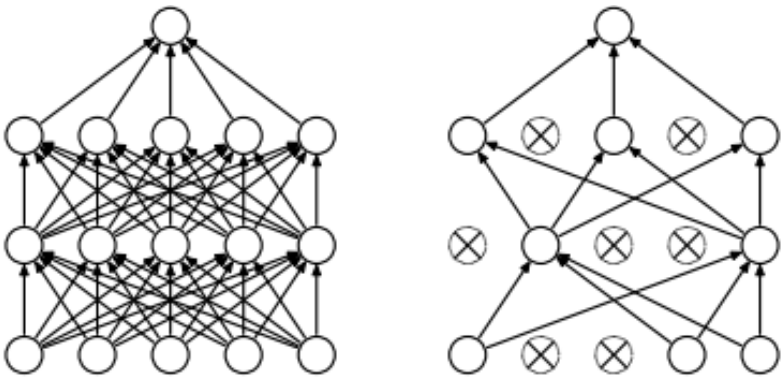
For the Attention U-Net GAN model, satisfying results were achieved. The implementation of attention aimed to enhance the extraction of key features, in order to colorize the images more accurately. How-

ever, the relatively limited size of the STL10 dataset posed a constraint to the effectiveness of the model. Expanding the dataset or using a more diverse set of images could further improve the performance and generalization of the model.

9

New Methodology of deep CNN

As the loss curve results is inadequate with regard to multinomial cross entropy loss method and the Euclidean loss method, the algorithm and CNN model probably have some weaknesses because my curve is obviously overfitting. The training curve is reduced smoothly. However, after a very short and unnoticeable drop, the loss curve of validation began to rise, i think the reson is that Deep CNN models can capture complex patterns thanks to their many nonlinear layers. But with too little training data, they tend to learn noise from figures as if it were signal—capturing relationships that don't hold in new data from the same source—thus overfitting.



(a) Standard Neural Net (b) After applying dropout.

Figure 1: Dropout Neural Net Model. **Left**: A standard neural net with 2 hidden layers. **Right**: An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Figure 9.1: **Dropout neural network.**

F ollowing Srivastava et al. [Sri+14], we use the term *dropout* to denote the random omission of units (both input and hidden) during training. At each update, a subset of units is sampled and *temporarily removed* from the computation graph together with all incident incoming and outgoing connections, producing a thinner subnetwork for that iteration (see Fig. 9.1). The subset is resampled independently at each step.

S tarting from the baseline encoder-only *TinyColorizationNet2* (three convolutional blocks with downsampling at stages 2 and 3, each using the sequence `Conv` → `BN` → `ReLU`, followed by a 1×1 classification

head), we insert stochastic regularization in the form of dropout. Specifically, we place *spatial* dropout (`Dropout2d`) *after* each block, with increasing rates $p = \{0.25, 0.35, 0.40\}$ from shallow to deep layers, and an elementwise `Dropout` with $p = 0.30$ before the final 1×1 classifier. An optional input dropout is kept disabled by default. All dropout layers are positioned *after* batch normalization so that BN statistics are computed on full activations. The head remains linear (no activation/BN on the logits) and now outputs $Q = 313$ channels corresponding to quantized ab color bins. This modification reduces feature-map co-adaptation and acts as implicit model averaging, thereby mitigating the overfitting observed with the original architecture. *TinyColorizationNet* is the model of deep CNN.

T he main difference of the multinomial cross entropy loss method and Euclidean loss method is the output of the model. The model belong to multinomial cross entropy loss method is CNN-1 and The model belong to Euclidean loss method is CNN-2. The CNN-1 output is a 4-dimensional matrix. Here, B is the batch size (number of input images in the mini-batch);The 313 channels are logits over the quantized ab color bins; applying a softmax along the channel dimension yields a per-pixel probability distribution over bins; H' and W' are the spatial height and width at the output resolution. The CNN-2 output is a 4-dimensional matrix as well, but the second dimension 2 values is the two channels which are the continuous chrominance values a and b at each pixel be the ground truth. Other dimensions all same.

After that, the results of the loss curve are more reasonable, but the accuracy still do not make sense, especially for the validation curve in the CNN-1.

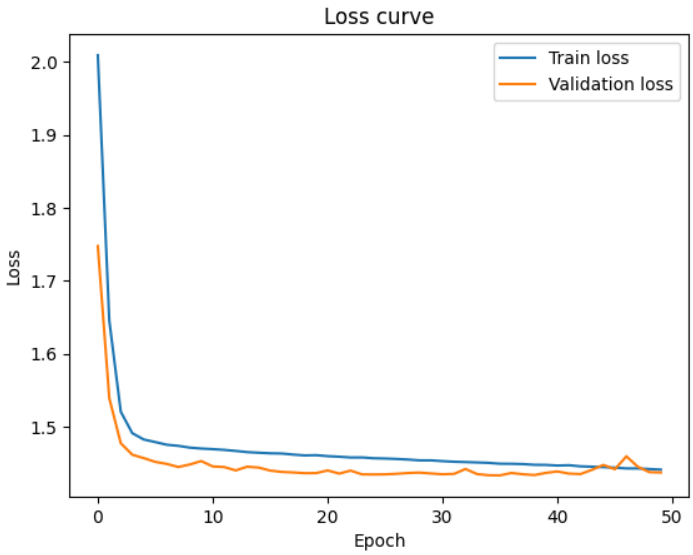


Figure 9.2: CNN-1 loss: blue = train, orange = validation.

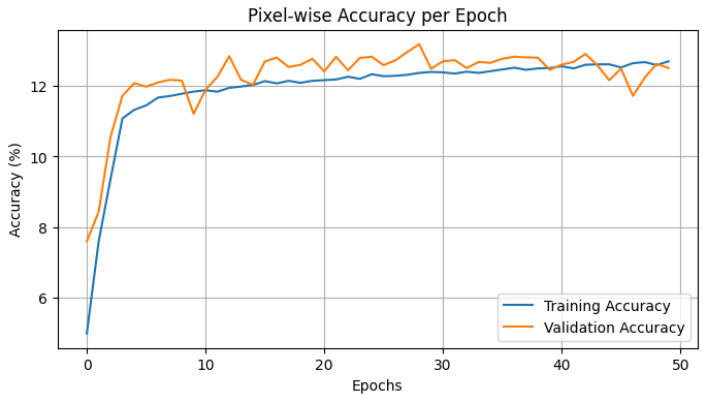


Figure 9.3: CNN-1 accuracy: blue = train, orange = validation.

I am checking back to my code again. i find that when I compare the probability distribution over bins from logits. I previously reported top-1 pixel accuracy (only the most probable bin must match), which is very strict and yielded low values. I now report top-5 pixel accuracy—counting a pixel as correct if the ground-truth bin appears among the five highest-probability bins—to obtain a more stable validation metric. Here the bin means A lookup table that stores the centers of the quantized color bins in the CIE Lab ab plane.

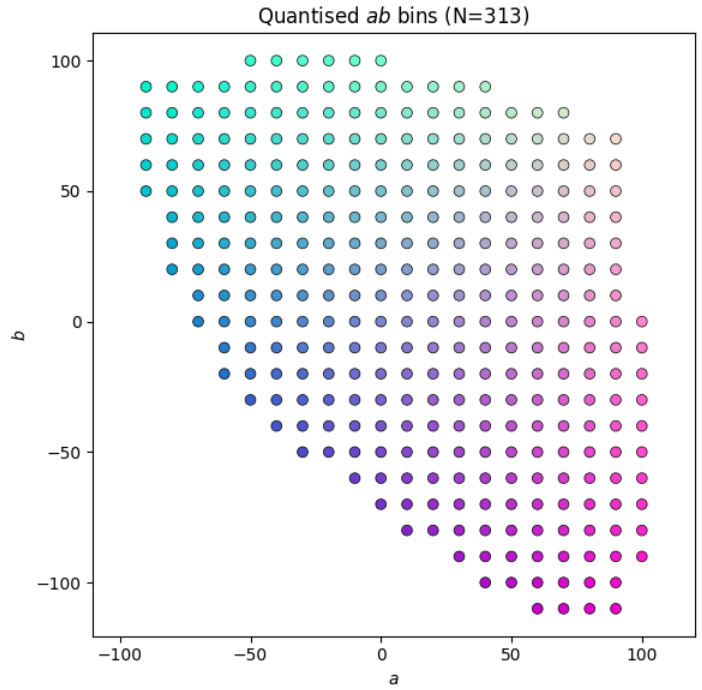


Figure 9.4: CNN-1 loss: blue = train, orange = validation.

Figure 9.4 shows the $Q = 313$ quantised chrominance bin centres in the CIE Lab ab-plane. Each point $\boldsymbol{\mu}_q = (a_q^*, b_q^*) \in \mathbb{R}^2$ is a lookup entry in

$$\texttt{ab_bins} = \{\boldsymbol{\mu}_q\}_{q=1}^{Q} \in \mathbb{R}^{Q\times 2}.$$

This lookup table discretises the continuous ab space and is used to turn a ground-truth pixel colour $\mathbf{c} = (a^*, b^*)$ into a sparse, probabilistic label ("soft target") over the Q bins.

Given $\mathbf{c}$, we select its k nearest bin centres under Euclidean distance,

$$\mathcal{N}_k(\mathbf{c}) = \underset{q \in \{1,\dots,Q\}}{\operatorname{argtopk}} \Big(- \|\mathbf{c} - \boldsymbol{\mu}_q\|_2^2 \Big),$$

and define Gaussian weights on that neighbourhood,

$$\tilde{T}_q(\mathbf{c}) = \begin{cases} \exp\big(-\frac{1}{2\sigma^2} \|\mathbf{c} - \boldsymbol{\mu}_q\|_2^2\big), & q \in \mathcal{N}_k(\mathbf{c}), \\ 0, & \text{otherwise.} \end{cases} \qquad T_q(\mathbf{c}) = \frac{\tilde{T}_q(\mathbf{c})}{\sum_{r=1}^{Q} \tilde{T}_r(\mathbf{c})}.$$

For an image batch this yields a tensor of soft targets $T \in \mathbb{R}^{B \times Q \times H \times W}$, where $T_{b,q,i,j} = T_q\big(\mathbf{c}_{b,i,j}\big)$. Intuitively, the figure visualises the support points $\{\boldsymbol{\mu}_q\}$ over which each pixel's ab value is softly encoded into a Q-dimensional distribution concentrated on its k nearest bins.

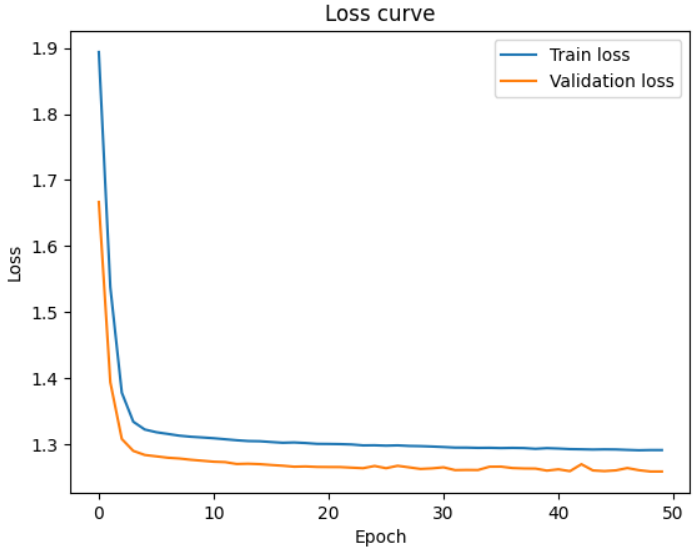


Figure 9.5: CNN-1 loss: blue = train, orange = validation.

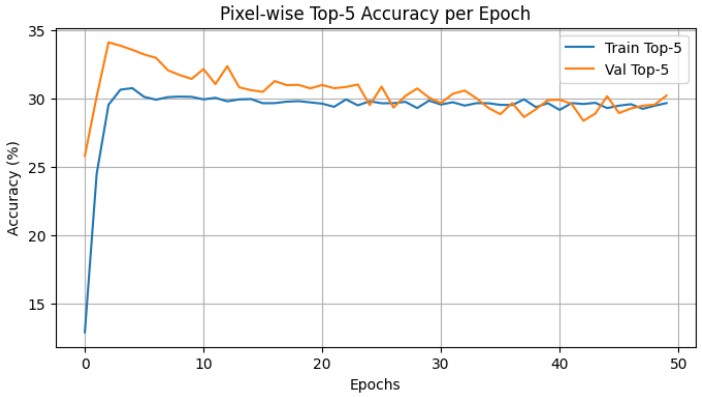


Figure 9.6: CNN-1 accuracy for top 5 pixel: blue = train, orange = validation.

After top-5 pixel accuracy—counting a pixel, the accuracy has improved from 12% to 35%, but the issue, which the algorithm still cannot running in long epochs.

in the CNN-2, as the Euclidean loss function is not continuous for pixel to pixel, the accuracy naturally jitters, so i try use RMSE methed to be a standard to evaluate model performance. The root mean squared error (RMSE) quantifies the typical magnitude of prediction error as the square root of the mean of squared residuals. the Lab ab value has two channels, so I need combine error from a value and error from b value.

$$\mathbf{c}_i = \begin{bmatrix} a_i \\ b_i \end{bmatrix}, \qquad \hat{\mathbf{c}}_i = \begin{bmatrix} \hat{a}_i \\ \hat{b}_i \end{bmatrix}.$$

$$\delta_i \;=\; \|\hat{\mathbf{c}}_i - \mathbf{c}_i\|_2 \;=\; \sqrt{(\hat{a}_i - a_i)^2 + (\hat{b}_i - b_i)^2}\,.$$

$$\mathrm{RMSE}_{\mathrm{ab}} \;=\; \sqrt{\frac{1}{N}\sum_{i=1}^{N} \delta_i^2} \;=\; \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left[(\hat{a}_i - a_i)^2 + (\hat{b}_i - b_i)^2\right]}\,.$$

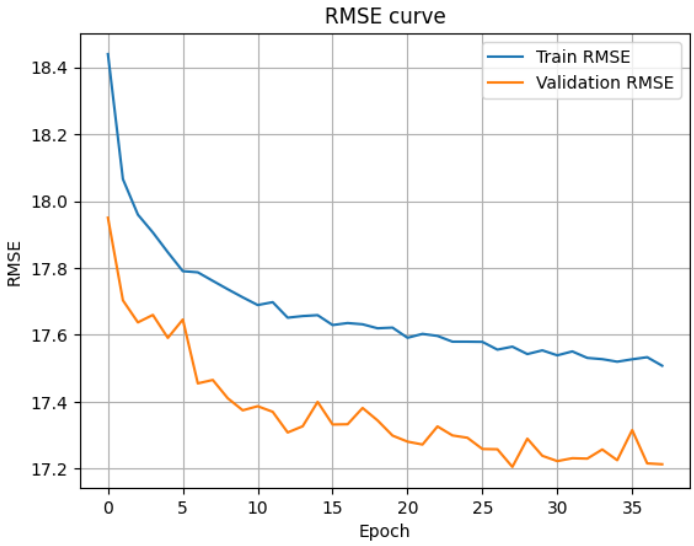


Figure 9.7: CNN-2 loss: blue = train, orange = validation.

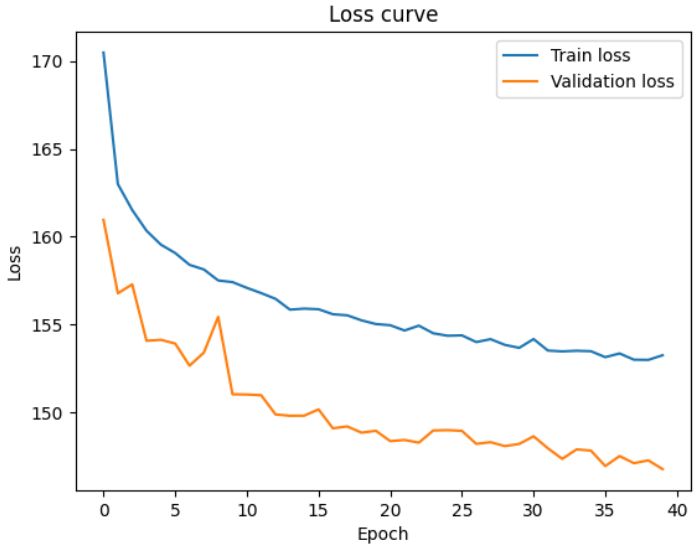


Figure 9.8: CNN-2 RMSE: blue = train, orange = validation.

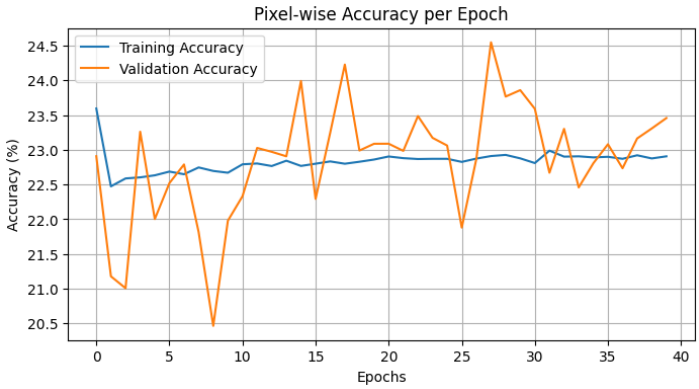


Figure 9.9: CNN-2 accuracy: blue = train, orange = validation.

I know the RMSE essentially an extension of the loss curve. However, I really don't have the method to improve the accuracy.

In my CNN-1 case, I found that after some epochs the RMSE changed very slowly essentially plateauing, so I used early stopping to end training early. Early stopping is a regularization technique for iterative optimization (e.g., gradient descent) that halts training once the validation error stops improving, thereby preventing the model from overfitting to noise and dataset characteristics.

I monitor the validation RMSE m_t at each epoch t. Let ϵ be the smallest improvement we care about and P the patience (how many epochs we can wait without improvement). We also keep the best score $m^\star$ seen so far and a counter c.

- If $m_t < m^\star - \epsilon$, we have a real improvement: set $m^\star \leftarrow m_t$, reset $c \leftarrow 0$, and save the model.
- Otherwise, no meaningful improvement: increase $c \leftarrow c + 1$.
- Stop training when $c \geq P$ (the curve has plateaued).

- If `restore_best` is on, reload the checkpoint at $m^\star$ so evaluation uses the best generalizing weights.

This keeps training going while it is truly getting better, and stops once the validation metric stops improving.

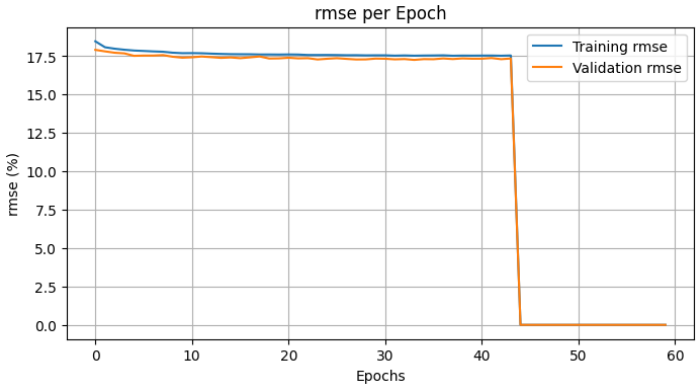


Figure 9.10: CNN-2 RMSE early stop: blue = train, orange = validation.

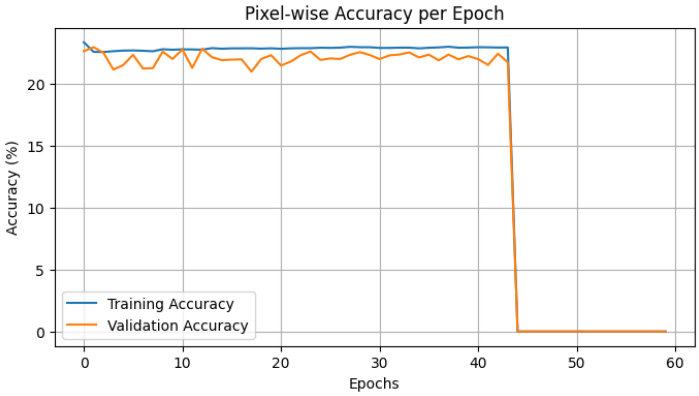


Figure 9.11: CNN-2 accuracy early stop: blue = train, orange = validation.

Bibliography

[CNL11] Coates, Adam, Andrew Ng, and Honglak Lee (2011). “An Analysis of Single-Layer Networks in Unsupervised Feature Learning”. In.

[GS] G., Vishnu and Srinivas S. (n.d.). “All about Structural Similarity Index (SSIM): theory & code in PyTorch”. In: (). Accessed 2025-06-07.

[NNE18] Nazeri, Kamyar, Eric Ng, and Mehran Ebrahimi (2018). “Image Colorization using Generative Adversarial Networks”. In: *arXiv preprint arXiv:1803.05400*. URL: https://arxiv.org/abs/1803.05400.

[RFB15] Ronneberger, Olaf, Philipp Fischer, and Thomas Brox (2015). “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In.

[SZ14] Simonyan, Karen and Andrew Zisserman (2014). “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *arXiv preprint arXiv:1409.1556*.

[Sri+14] Srivastava, Nitish et al. (2014). “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.56, pp. 1929–1958.

[Wan] Wang, Zhou (n.d.). “SSIM Index – official project page”. In: (). Accessed 2025-06-07.

[Wan+04] Wang, Zhou et al. (2004). “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: *IEEE Trans. Image Processing* 13.4, pp. 600–612. URL: https://www.cns.nyu.edu/pub/eero/wang03-reprint.pdf.

[ZIE16] Zhang, Richard, Phillip Isola, and Alexei A. Efros (2016). “Colorful Image Colorization”. In: pp. 649–666. DOI: 10.1007/978-3-319-46487-9_40. URL: http://richzhang.github.io/colorization/.